# Retriever-final-class

29 July 2026 14:34

**RAG Pipeline Intro.**

Documents
↓
Parsing / Loading
↓
Chunking
↓
Embeddings
↓
Vector Store
↓
Retriever
↓
LLM
↓
Final Answer

Prompting

**What is retriever**

Vector DB | store

A retriever takes a user query and returns the most relevant documents or chunks from a knowledge source.

A retriever normally does not generate the final answer itself; it collects the relevant context required to generate the ans wer.

A retriever is a component that:

Takes a user query
↓
Searches the knowledge source
↓
Returns the most relevant documents

A retriever's job is to find relevant information based on the user's question.

User question
↓
Retriever searches the stored documents
↓
Returns the most relevant chunks
↓
The LLM uses those chunks to generate the answer

**Simple example**

Suppose a vector database contains 1,000 chunks extracted from PDF documents.
The user asks:

user ques

What is semantic chunking?
The retriever does not send all 1,000 chunks to the LLM. It searches the database and returns only the relevant chunks:

Chunk 12 → Definition of semantic chunking
Chunk 48 → Example of semantic chunking
Chunk 91 → Advantages of semantic chunking
These relevant chunks are then passed to the LLM.

Context
Retrieve data
Ranked data

**Simple analogy**

Think of a retriever as a librarian:

User = Student
Documents = Library books
Retriever = Librarian
LLM = Teacher
The student asks a question. The librarian finds the most relevant books or pages and gives them to the teacher. The teacher reads those pages and generates the final answer.


RAG

Langchain
Llamaindex
haystack

95-99 %

**code reminder from langchain**

```
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={
        "k": 4,         # Final number of documents to return
        "fetch_k": 20    # Candidate documents considered by MMR
    }
)
```

```
# Retrieve relevant documents
documents = retriever.invoke(
    "What is a vector database?"
)
```

```
# Display the retrieved content and metadata
for document in documents:
    print(document.page_content)
    print(document.metadata)
    print("-" * 50)
```

context of user query

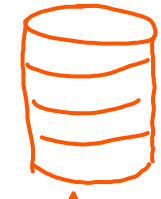
K=10/20/30

50%

.k = final number of results
filter = metadata restriction
score_threshold = minimum acceptable relevance score
fetch_k = number of candidates fetched before MMR
lambda_mult = balance between relevance and diversity in MMR

Cosine_Similarity
-1 to +1
no match
more similare (full match)

search_type → decides which search algorithm runs
similarity → returns the most similar chunks
similarity_score_threshold → returns only the chunks that pass the threshold
mmr → returns relevant and diverse chunks
search_kwargs → configures the selected search algorithm



Advance type of similarity search
CS
MMR
Fetch_k
lambda_mult

Similarity
Query
search_type
= Cosine
= Dot
= ED

### Similarity Search Methods

**1. Cosine Similarity**

Measures the **angle/direction similarity** between two vectors.

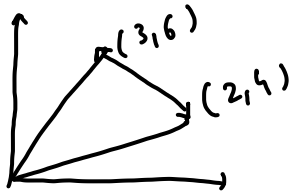
$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Higher score = More similar
Most commonly used for text embeddings.

**2. Euclidean Distance — L2**

Measures the **straight-line distance** between two vectors.

$$d(A, B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$$




$\sqrt{(4-6)^2 + (2-3)^2}$

$\sqrt{(-2)^2 + (-1)^2} = \sqrt{4+1}$

$= \sqrt{5}$

Smaller distance = More similar

**3. Dot Product / Inner Product**

- Multiplies corresponding vector values and adds them.

$$A \cdot B = \sum_{i=1}^{n} A_i B_i$$ (-∞ to +∞) → normalize value

-1 to +1 ← cosine similarity

Higher score = More similar

With normalized vectors, dot product becomes equivalent to cosine similarity.

**Easy Summary**

| Method | Best result |
|---|---|
| Cosine similarity | Highest score |
| Euclidean distance | Lowest distance |
| Dot product | Highest score |

→ MMR

These are the three primary similarity metrics commonly supported by vector databases such as Pinecone.

## Metadata Filtering

A retriever should not rely only on semantic similarity. It should also support structured constraints using document metadat a.

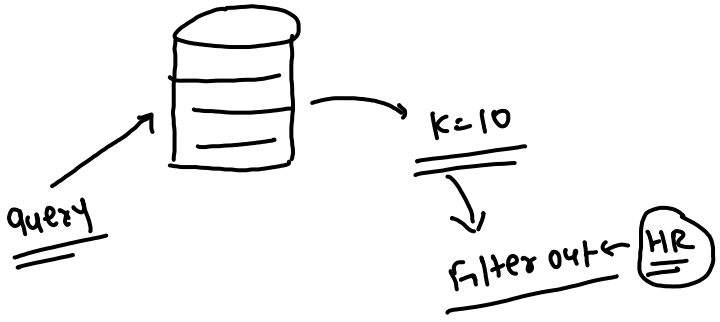
Metadata helps the retriever limit the search to documents that satisfy specific conditions such as department, year, documen t type, source, user role, or tenant.

### Example Metadata

```
metadata = {
   "department": "HR",
   "year": 2026,
   "document_type": "policy",
   "access_role": "manager"
}
```

### User Query

Show the HR leave policy for 2026.




### Metadata Filter

```
filter = {
   "department": "HR",
   "year": 2026
}
```

The retriever will search only the documents whose metadata matches:

department = HR

year = 2026

It will ignore documents from other departments or years, even when their content is semantically similar to the query.

## Common Types of Metadata Filters

- **Exact-match filters**
  Match an exact metadata value.

  ```
  {"department": "HR"}
  ```
- **Range filters**
  Match values within a range.

  ```
  {"year": {"$gte": 2024, "$lte": 2026}}
  ```
- **Boolean filters**
  Combine multiple conditions using AND, OR, or NOT.

  ```
  {
    "$and": [
      {"department": "HR"},
      {"year": 2026}
    ]
  ```

}

- **Date filters**
  Retrieve documents created or updated within a specific date range.
- **Department filters**
  Restrict retrieval to departments such as HR, Finance, Legal, or Engineering.
- **Document-type filters**
  Restrict retrieval to policies, reports, invoices, manuals, or contracts.
- **Source filters**
  Search only selected PDFs, websites, databases, or repositories.
- **Tenant filters**
  Ensure that users can retrieve documents only from their own organization or tenant.
- **Role-based access filters**
  Restrict documents according to roles such as employee, manager, HR, or administrator.
- **Pre-filtering and post-filtering**
  Decide whether metadata restrictions are applied before or after the retrieval operation.

Vector databases commonly support metadata restrictions during retrieval. These filters reduce the search space and help retu rn only relevant and permitted documents.

The exact filter syntax may differ between vector databases such as Chroma, Pinecone, Qdrant, Weaviate, and Elasticsearch.

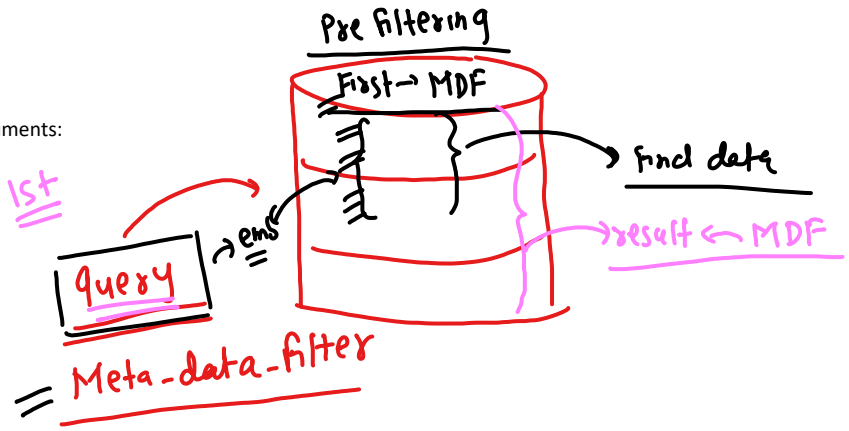
Pre-filtering:
Filter first → Search later

Post-filtering:
Search first → Filter later

First filter then perform vector search

**Pre-filtering**
Pre-filtering means applying metadata conditions **before** running vector or keyword search.

All stored documents
↓
Apply metadata filter
↓
Allowed documents only
↓
Similarity or keyword search
↓
Final results




**Example**
Suppose the vector database contains 10,000 documents:

HR documents = 1,000
Finance documents = 3,000
Engineering documents = 4,000
Legal documents = 2,000
The user asks:

Show the HR leave policy for 2026.
The filter is:

```
filter = {
  "department": "HR",
  "year": 2026
}
```

With pre-filtering:

10,000 documents
↓
Filter department = HR and year = 2026
↓
Only 150 permitted documents remain
↓
Similarity search runs on those 150 documents
↓
Most relevant HR leave-policy chunks are returned

**Code Example**

```
retriever = vector_store.as_retriever(
  search_type="similarity",
  search_kwargs={
    "k": 4,
```

```
    "filter": {
      "department": "HR",
      "year": 2026
    }
  }
)
documents = retriever.invoke(
  "Show the HR leave policy for 2026."
)
```

**Advantages of Pre-filtering**

- Searches a smaller document set
- Reduces irrelevant results
- Improves security
- Supports tenant isolation
- Prevents unauthorized documents from entering the candidate list
- Can improve retrieval speed

Pre-filtering is generally preferred for strict authorization because restricted documents are excluded before retrieval begins.

**Post-filtering**

Post-filtering means running retrieval first and applying metadata conditions afterward.

All stored documents
↓
Similarity or keyword search
↓
Top candidate documents  K = 10
↓
Apply metadata filter
↓
Final allowed results

**Example**

Suppose the retriever first returns the top five semantically similar documents:

Result 1 → Finance leave policy, 2026
Result 2 → HR leave policy, 2025
Result 3 → Legal leave guideline, 2026
Result 4 → HR leave policy, 2026
Result 5 → Engineering leave policy, 2026
Now the filter is applied:

```
filter = {
  "department": "HR",
  "year": 2026
}
```

After post-filtering, only one result remains:

Result 4 → HR leave policy, 2026
The retriever originally fetched five documents, but four were removed after retrieval.

**Post-filtering Example**

```
documents = vector_store.similarity_search(
  "Show the HR leave policy for 2026.",
  k=5
)
filtered_documents = [
  document
  for document in documents
  if document.metadata.get("department") == "HR"
  and document.metadata.get("year") == 2026
]
```

**Limitations of Post-filtering**

- It may return too few final results
- Relevant permitted documents may never enter the initial top-k
- Unauthorized documents may enter the intermediate candidate set
- It is less suitable for strict access control
- A larger initial k may be required

For example:

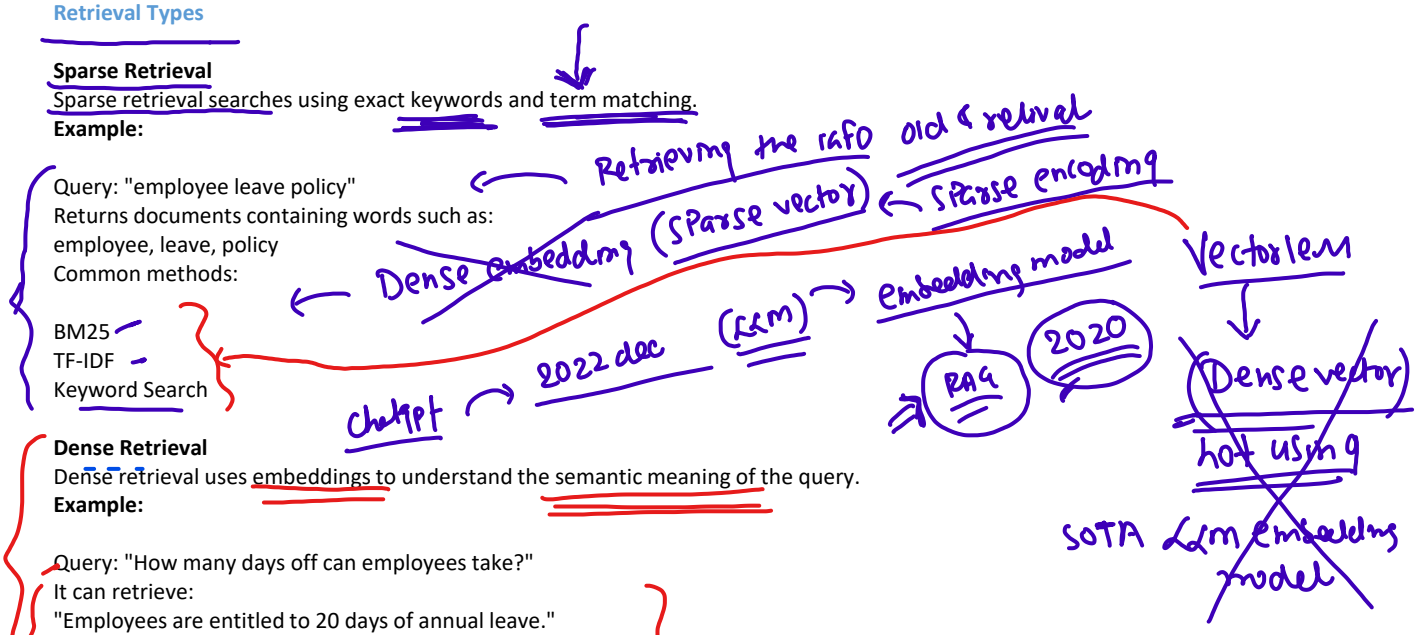
Initial retrieval returns top 5
↓
4 results fail the filter
↓
Only 1 final result remains
Even though more valid HR documents may exist in the database, they may not have appeared in the original top five.

## Retrieval Types

**Sparse Retrieval**
Sparse retrieval searches using exact keywords and term matching.
**Example:**

Query: "employee leave policy"
Returns documents containing words such as:
employee, leave, policy
Common methods:

BM25
TF-IDF
Keyword Search




**Dense Retrieval**
Dense retrieval uses embeddings to understand the semantic meaning of the query.
**Example:**

Query: "How many days off can employees take?"
It can retrieve:
"Employees are entitled to 20 days of annual leave."
The exact words may be different, but the meaning is similar.

**Hybrid Retrieval**
Hybrid retrieval combines sparse and dense retrieval.

Keyword/BM25 Search (sparse)
+
Vector Search (dense)
↓
Combined Results
**Example:**

Query: "HR leave policy 2026"
Sparse retrieval matches exact terms such as:

HR
leave policy
2026
Dense retrieval finds semantically similar content such as:

employee annual vacation guidelines
Hybrid retrieval combines both results for better accuracy.

**Query Transformation**
Query transformation improves the user's original query before retrieval so the system can find more relevant information. It may rewrite the query, add related terms, or break a complex query into smaller questions.

1. **Query Rewriting**

Query rewriting converts an unclear, incomplete, or conversational query into a clearer standalone search query.
**Example:**

Original query:
"What did he say about it?"
Rewritten query:
"What did the CEO say about the 2026 acquisition?"
This is especially useful in conversational RAG, where the current question depends on previous messages.

Original query
↓
Clearer standalone query
↓
Retriever

**2. Query Expansion**
Query expansion adds synonyms, related terms, acronyms, spelling variations, or alternative phrases to the original query.
**Example:**

Original query:
"employee leave policy"
Expanded query:
"employee leave policy OR vacation policy OR annual leave guidelines"
Another example:

Original term:
"car"
Expanded terms:
"car, automobile, vehicle"
Query expansion broadens the search and helps retrieve documents that use different words for the same concept.

Original query
↓
Add related terms or synonyms
↓
Broader retrieval

**3. Query Decomposition**
Query decomposition breaks a complex question into smaller and simpler sub-questions.
**Example:**

Original query:
"Compare the revenue of Company A and Company B in 2025
and explain why their growth rates were different."
It can be decomposed into:

Sub-query 1:
What was Company A's revenue in 2025?
Sub-query 2:
What was Company B's revenue in 2025?
Sub-query 3:
What factors affected Company A's growth?
Sub-query 4:
What factors affected Company B's growth?
The system retrieves information for each sub-query and combines the results to answer the original question. Query decomposition is useful for comparison, multi-hop, and complex questions.

Complex query
↓
Multiple smaller sub-queries
↓
Retrieve evidence for each query
↓
Combine the results

Reranking
Reranking is a second-stage process that takes documents returned by an initial retriever, scores them more accurately against the user query, and reorders them so that the most relevant documents are sent to the LLM.

## Benefits of Reranking

Reranking can:

- Improve the ordering of retrieved documents
- Remove weak candidates using a relevance threshold
- Reduce irrelevant context sent to the LLM
- Reduce unnecessary input tokens
- Improve evidence quality
- Work on results from sparse, dense, or hybrid retrieval

Official production implementations such as Elasticsearch and Cohere accept an initial candidate set and reorder it according to query relevance; Elasticsearch also supports candidate-window size, score thresholds, filters, and chunk-level rescoring for long documents.

## Limitation

Reranking adds:

- Additional latency
- Additional computation
- Additional inference cost

Therefore, it should normally be applied only to a limited candidate set —not to every document in the database.

Reranking is a **second-stage retrieval process** that takes an initial set of retrieved documents, calculates a more accurate relevance score for each document, and rearranges them from most relevant to least relevant.

User Query
↓
Initial Retriever
(BM25, Vector Search, or Hybrid Search)
↓
Top Candidate Documents
↓
Reranker
↓
Reordered by Relevance
↓
Top Documents Sent to the LLM

## Why Is Reranking Required?

The initial retriever must search thousands or millions of documents quickly. Therefore, it normally uses a fast retrieval me thod such as:

- BM25
- Vector similarity search
- Hybrid retrieval

Fast retrieval provides good candidates, but their original order may not be perfectly accurate.
A reranker applies a more powerful model only to this smaller candidate set. This creates a practical balance:

Initial Retrieval → Fast and broad
Reranking → Slower but more accurate
Production search systems use this multi-stage architecture because an expensive ranking model can be applied to a small candidate set rather than the entire document collection.

## Simple Example

Suppose the user asks:

How many days of annual leave do employees receive?
The initial retriever returns these candidates:

1. Remote Work Policy
2. Sick Leave Policy
3. Annual Leave Policy
4. Leave Carry-Forward Policy
5. Employee Attendance Policy

These documents are related to employees and leave, but the most useful document is not ranked first.
The reranker evaluates every candidate against the original query:

Annual Leave Policy → 0.95
Leave Carry-Forward Policy → 0.76
Sick Leave Policy → 0.31
Employee Attendance Policy → 0.19
Remote Work Policy → 0.08
The reranker then produces a better order:

1. Annual Leave Policy
2. Leave Carry-Forward Policy
3. Sick Leave Policy
4. Employee Attendance Policy
5. Remote Work Policy

Only the highest-ranked documents are passed to the LLM.

## How a Cross-Encoder Reranker Works

A standard embedding retriever usually encodes the query and documents separately:

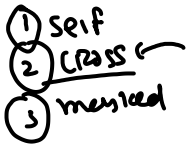
Query → Query Vector
Document → Document Vector

↓

Similarity Score

Because document vectors can be created and stored in advance, this approach is fast enough to search large collections.

A cross-encoder reranker processes the query and one candidate document **together**:




[Query + Candidate Document]

↓

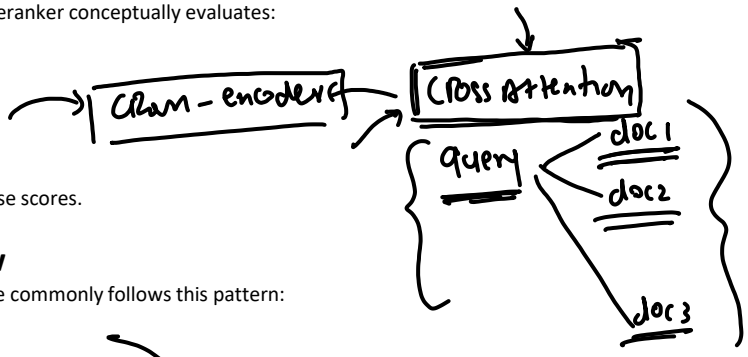
Cross-Encoder

↓

Relevance Score

This joint processing allows the model to examine detailed relationships between the words in the query and the document. How ever, every query-document pair must be processed separately, making cross-encoders more computationally expensive than first-stage vector retrieval.

For 20 candidate documents, the reranker conceptually evaluates:




Query + Document 1 → Score

Query + Document 2 → Score

Query + Document 3 → Score

...

Query + Document 20 → Score

It then sorts the documents by these scores.

## Real Production Flow

A practical production RAG pipeline commonly follows this pattern:


1. User submits a query

↓

2. Apply metadata and security filters

↓

3. Retrieve a broad candidate set
   using BM25, vector, or hybrid search

↓

4. Send the candidate set to a reranker

↓

5. Calculate query-document relevance scores

↓

6. Sort candidates by reranker score

↓

7. Apply an optional minimum-score threshold

↓

8. Send the best documents to the LLM

For example:

1,000,000 stored chunks

↓

Retriever selects 50 candidates

↓

Reranker reorders those 50 candidates

↓

Top 5 chunks are passed to the LLM

The values 50 and 5 are only examples. In production, candidate count and final result count are tuned according to accuracy, latency, model limi ts, token budget, and cost. Elasticsearch exposes this candidate window as rank_window_size, and reranking services accept a query plus a candidate document list and return relevance-ranked results.

## Was Reranking Introduced for RAG?

No. Reranking existed in **information retrieval and search systems before modern RAG**.

Traditional search systems already used multi-stage or cascade ranking:

Fast candidate generation

↓

More accurate ranking stages

↓

Final search results

The 2011 work *A Cascade Ranking Model for Efficient Ranked Retrieval* formalized a multi-stage ranking architecture designed to balance search effectiveness and computational efficiency.

In 2019, *Passage Re-ranking with BERT* demonstrated that BERT could be adapted to score query-passage pairs and substantially improve passage-ranking performance. This work helped popularize transformer-based semantic reranking, but it did not invent the general reranking concept.

RAG later adopted the same established idea:

Search documents first

↓

Rerank the candidates

↓
Give the best evidence to the LLM

## Reranking vs Retrieval

| Retrieval | Reranking |
|---|---|
| Searches the complete collection | Processes only retrieved candidates |
| Optimized for speed and recall | Optimized for ranking precision |
| Returns an initial candidate set | Reorders that candidate set |
| Commonly uses BM25 or embeddings | Commonly uses a cross-encoder or another ranking model |
| First stage | Second stage |

Retriever finds possible documents.
Reranker decides which of those documents are most relevant.

## Reranking vs Reciprocal Rank Fusion

They are not the same.

### Reciprocal Rank Fusion

RRF combines ranked lists produced by multiple retrievers:

BM25 Results ──┐
├── RRF → Combined Ranking
Vector Results ──┘

RRF mainly uses the **positions of documents in the input rankings**. It does not need to jointly understand every query-document pair.

### Semantic Reranking

A semantic reranker evaluates the relationship between the query and each candidate document:

Combined Candidates
↓
Query-Document Model
↓
New Relevance Scores
↓
Final Ranking

Therefore, a complete hybrid production pipeline may use both:

BM25 Search + Vector Search
↓
RRF
↓
Cross-Encoder Reranker
↓
Best Context for LLM

## Main Reranking Approaches

### 1. Pointwise Reranking

Each candidate is scored independently:

Query + Document A → 0.91
Query + Document B → 0.67
Query + Document C → 0.22
The documents are sorted by score.

### 2. Pairwise Reranking

The model compares two documents and determines which is more relevant:

For this query:
Document A or Document B?

### 3. Listwise Reranking

The model considers several candidates together and produces an ordered list.

Input:
Document A, B, C, D
Output:
C, A, D, B
The 2019 multi-stage BERT-ranking work describes pointwise monoBERT and pairwise duoBERT models arranged in a multi-stage ranking architecture.

### Multimodal Retriever

A multimodal retriever retrieves relevant information across different modalities, such as text and images. It uses multimoda l embedding models to represent compatible modalities in a shared vector space, allowing text-to-image, image-to-text and image-to-image similarity search.

Text-to-Image

Text Query
↓
Text Encoder
↓
Shared Embedding Space
↓
Search Image Vectors
↓
Relevant Images

Image-to-Image

Image Query
↓
Image Encoder
↓
Image Embedding Space
↓
Search Image Vectors
↓
Similar Images